



PROJECT REPORT

Campus Facility & Event Resource Booking System

An Object-Oriented Java Application for University Facility Coordination and Thread-Safe Concurrency Management

Submitted in partial fulfilment of the requirements for the course
Programming in Java (CSE2006)

Candidate Name: Narsingh Vishwakarma

Registration No: 24BEC10119

Program / Branch: B.Tech - Electronics and Communication
Engineering

Institution: VIT Bhopal University

Submission Date: September 2026

Table of Contents

1. Introduction	8. Implementation Details
2. Problem Statement	9. Execution Results & Traces
3. Functional Requirements	10. Testing Approach & Test Matrix
4. Non-Functional Requirements	11. Challenges Faced & Solutions
5. System Architecture	12. Learnings & Key Takeaways
6. Design Diagrams (UML & ER)	13. Future Enhancements
7. Design Decisions & Rationale	14. References

1. Introduction

In our college campus at VIT Bhopal, university infrastructure—ranging from the Sir Visvesvaraya Auditorium and high-performance computing labs to departmental conference rooms—is heavily utilized every week. Managing these resources across faculty lectures, guest seminars, and student club activities (like technical workshops and hackathons) is often challenging when done through manual permission slips, emails, or spreadsheets.

To address this practical campus issue, I designed and developed **CampusResourceHub**, a standalone Java console application. The project translates core concepts learned in the *Programming in Java* curriculum into a practical system. It leverages Object-Oriented Programming (inheritance hierarchies, polymorphism, interfaces), the Java Collections Framework, multi-threaded synchronization for conflict resolution, custom exception handling, Java 8+ Streams API for analytics, and file persistence.

2. Problem Statement

During college events, resource allocation typically encounters four main bottlenecks:

- **Simultaneous Booking Clashes:** Two organizers or faculty members might request the same venue for overlapping hours, resulting in double-bookings.
- **Priority Ambiguity:** High-priority university-level academic events or exams need a mechanism to take precedence over informal student club practice sessions.
- **Unmonitored Booking Quotas:** Without systemic constraints, a single student organization can reserve multiple venues simultaneously, depriving other clubs of available slots.
- **Lack of Real-Time Utilization Metrics:** Campus administration lacks visibility into which facilities have the highest demand and total hours used.

3. Functional Requirements

The system is organized into three major functional modules:

3.1 Module 1: User Roles & Quota Management

- **Role-Based Access:** Implements an abstract base `User` class with three concrete subclasses: `AdminUser`, `FacultyUser`, and `StudentClubUser`.
- **Dynamic Quota Constraints:** Enforces systemic booking limits based on role (e.g., max 3 active concurrent bookings for student clubs, 10 for faculty, unlimited for administrators).
- **Role Switching:** Allows switching active users at runtime to demonstrate access restrictions and permission enforcement.

3.2 Module 2: Resource Catalog & Thread-Safe Reservation Engine

- **Specialized Venue Modeling:** Models venues using an abstract `CampusResource` class extended by `Auditorium`, `ComputerLab` (tracking GPU workstations and OS), and `ConferenceRoom` (tracking smartboards and AV setups).
- **Time-Window Overlap Detection:** Implements a `TimeSlot` value object utilizing `java.time.LocalDateTime` to calculate temporal overlap with mathematical precision.
- **Thread-Safe Concurrency Locks:** Protects venue allocations using intrinsic monitor locks mapped per resource (`ConcurrentHashMap<String, Object>`) to eliminate race conditions.
- **Priority-Based Preemption:** When a faculty member books a slot already occupied by a student club, the system cancels the club booking with an audit note and assigns the slot to faculty.

3.3 Module 3: Streams Analytics, Audit Trail & CSV Export

- **Java Streams Analytics:** Computes real-time facility utilization using declarative stream operations (`filter`, `map`, `groupingBy`, `summingDouble`).
- **Audit Trail Ledger:** Records every booking, cancellation, and preemption action into a thread-safe log with timestamps.
- **CSV Report Export:** Exports historical reservation data into a standardized CSV file.

4. Non-Functional Requirements

- **Performance:** Fast slot search and clash validation (<10 ms) utilizing in-memory hash maps and localized locking.
- **Reliability & Concurrency Safety:** Guaranteed prevention of double-bookings even under simultaneous multi-threaded requests, validated using automated `CountDownLatch` stress testing.
- **Maintainability:** Clean modular design across 14 decoupled Java classes, interfaces, and custom exception types.
- **Explicit Exception Handling:** Domain-specific checked exceptions (`SlotConflictException`, `QuotaExceededException`, `ResourceNotFoundException`, `UnauthorizedAccessException`).
- **Ease of Execution:** Self-contained CLI runnable with standard `javac` and `java` without external build tools or databases.

5. System Architecture

The application uses a 4-tier layered architecture separating presentation, business logic, domain entities, and data persistence:

- **Presentation Layer:** CLI console menu (`Main.java`) and automated test runner (`TestRunner.java`).
- **Service Layer:** Business logic coordination (`BookingService`, `UserService`, `ResourceService`, `AnalyticsService`).
-
-

Domain Model Layer: OOP entities (User hierarchy, CampusResource hierarchy, Reservation , TimeSlot).

Persistence Layer: File I/O serialization (StorageManager) and CSV generation.

6. Design Diagrams

6.1 System Architecture Diagram

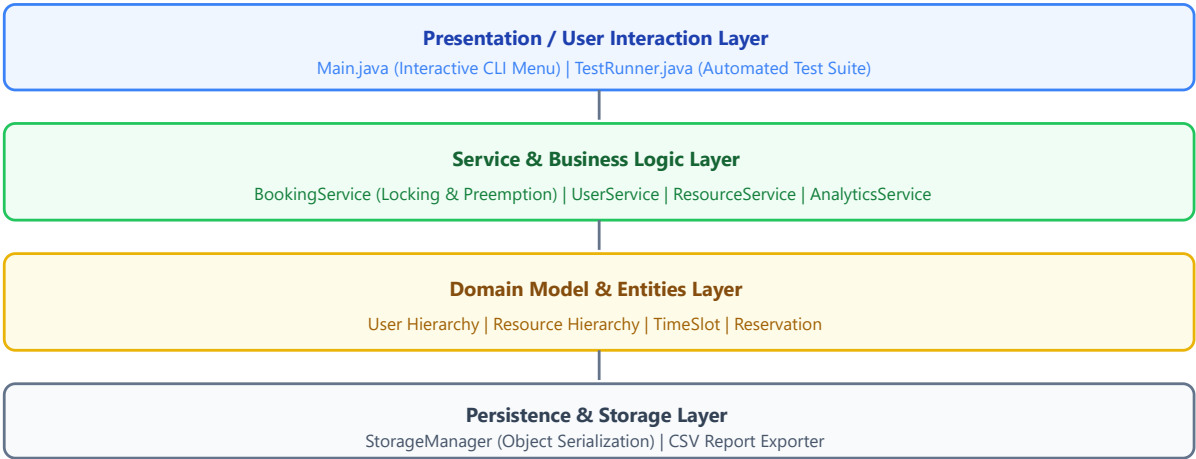


Figure 6.1: Four-Tier System Architecture of CampusResourceHub

6.2 Process Flow / Workflow Diagram

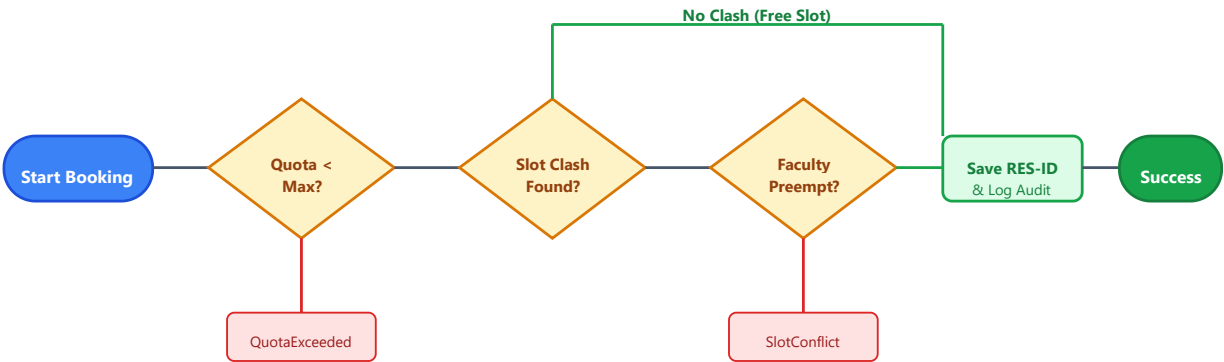
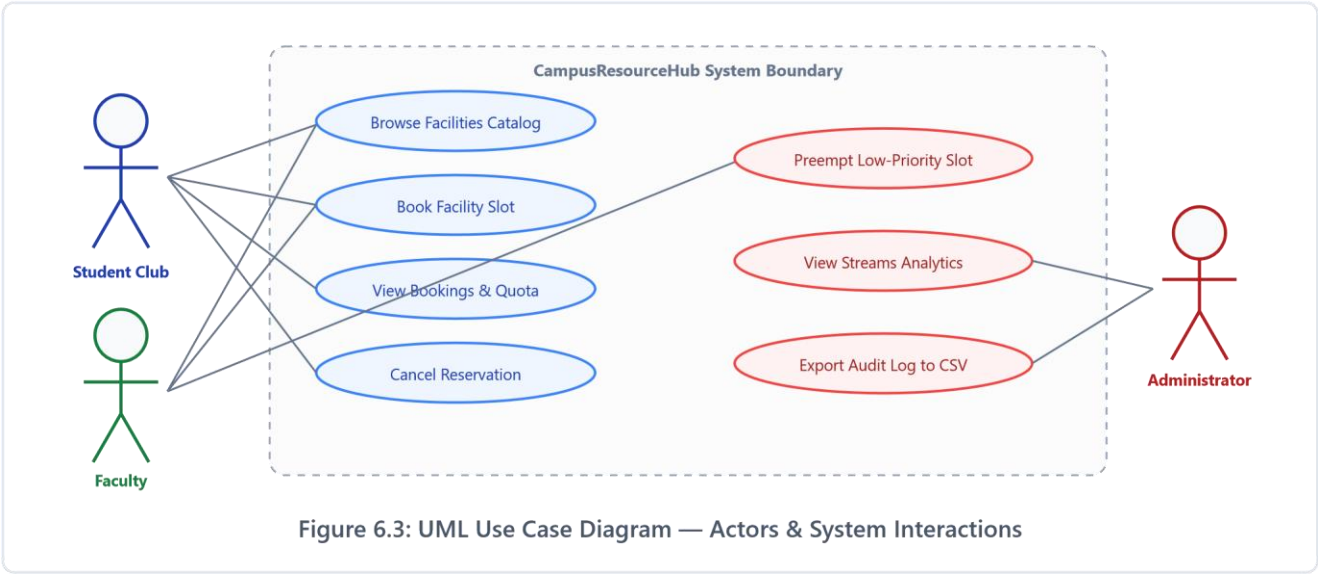
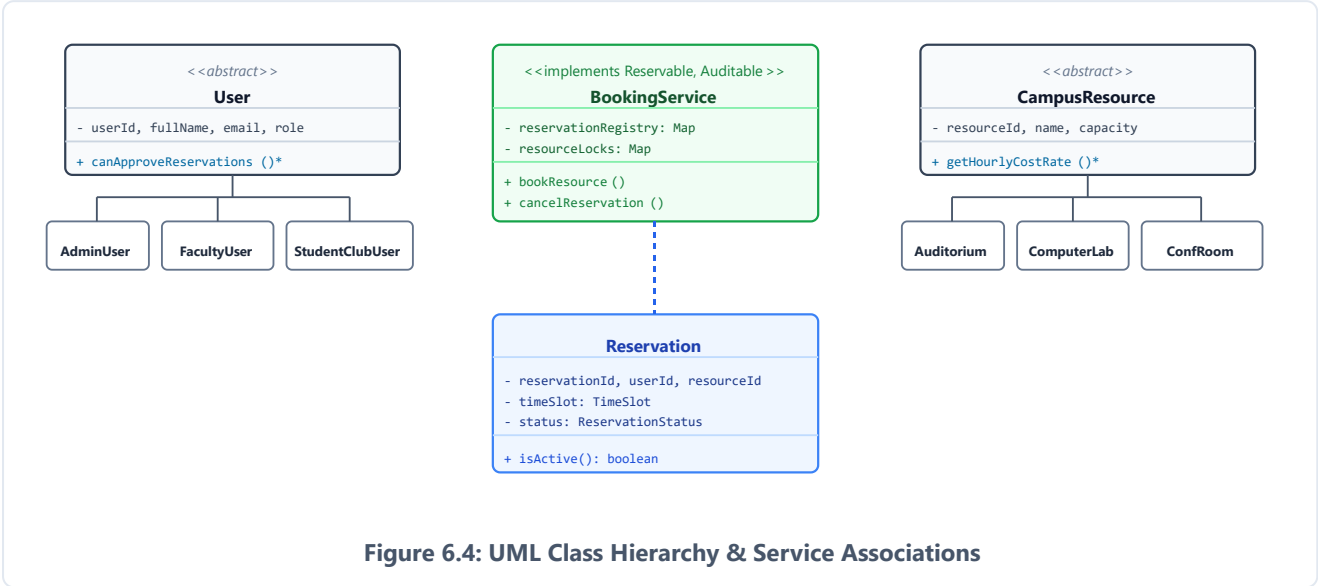


Figure 6.2: Reservation Workflow with Priority Preemption Logic

6.3 UML Use Case Diagram



6.4 UML Class Diagram



6.5 UML Sequence Diagram — Concurrent Race Condition

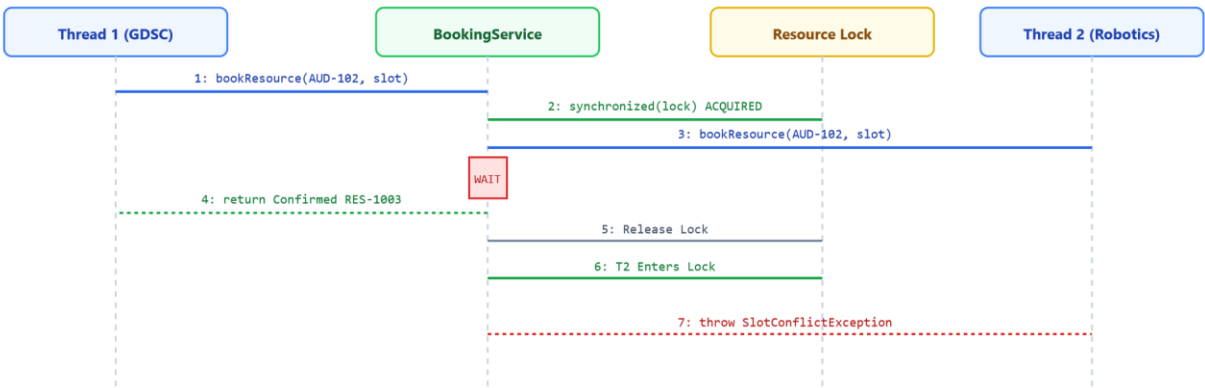


Figure 6.5: Sequence Diagram — Thread Synchronization on Competing Slot Requests

6.6 Entity Relationship (ER) Diagram

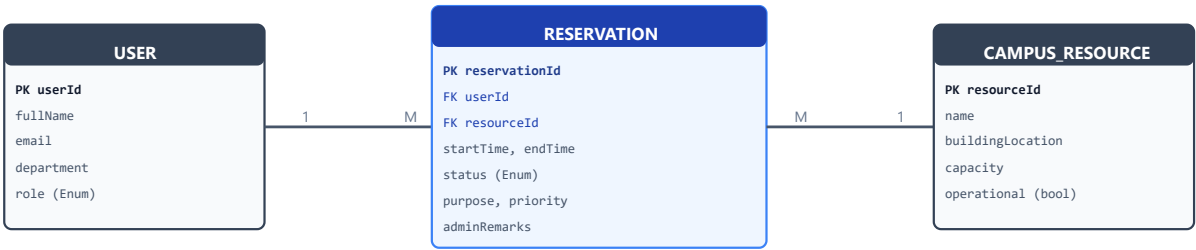


Figure 6.6: Entity-Relationship Schema with Foreign Key Cardinalities

7. Design Decisions & Rationale

Design Aspect	Chosen Strategy	Rationale
Lock Granularity	Striped per-resource monitor locks	A single global lock would serialize all reservations. Striping by resource ID allows concurrent bookings on different venues.
Temporal Logic	<code>java.time.LocalDateTime</code> with <code>TimeSlot</code>	Eliminates legacy <code>java.util.Date</code> bugs. Strict inequality ensures back-to-back bookings do not collide.
Interface Segregation	Reservable and Auditable	Separates booking logic from audit logging, enabling focused testing.
Custom Exceptions	Hierarchy rooted at <code>BookingException</code>	Enforces explicit handling so users receive clear error explanations.
Analytics Engine	Java 8+ Streams API	Concise declarative processing replaces boilerplate loops.

8. Implementation Details

The project consists of 14 well-structured classes under `com.vityarthi.booking`:

- `model/UserRole.java` : Enum specifying priority levels and max quotas. `model/User.java` : Abstract class holding identity fields. `model/AdminUser.java` , `FacultyUser.java` , `StudentClubUser.java` : Concrete user implementations. `model/CampusResource.java` : Abstract resource base with abstract methods `getResourceType()` and `getHourlyCostRate()` . `model/Auditorium.java` , `ComputerLab.java` , `ConferenceRoom.java` : Concrete venue models. `model/TimeSlot.java` : Value object with overlap logic:
- ```
public boolean overlapsWith(TimeSlot other) {
 return this.startTime.isBefore(other.endTime) && this.endTime.isAfter(other.startTime); }
```
- `model/Reservation.java` : Immutable record tracking status and audit remarks.
- `service/BookingService.java` : Thread-safe reservation engine with preemption.
- `service/AnalyticsService.java` : Streams aggregator and CSV exporter.
- `util/StorageManager.java` : Java Object Serialization persistence.
- `Main.java` : Interactive 10-option command-line console.

## 9. Execution Results & Run Traces

---

### 9.1 Interactive Main Menu

---

```
=====
CAMPUS FACILITY & RESOURCE BOOKING SYSTEM (CampusResourceHub)
Flipped Course Evaluation Project | Programming in Java
=====
>>> Current Active User: [STUDENT_CLUB] Rohan Mehta (club_gdsc) - Dept: CSE

1. Browse All Campus Facilities & Resources
2. Book a Resource / Facility Slot
3. View My Active Reservations & Quota
4. Cancel a Reservation
5. Switch User Account (Faculty / Club / Admin)
6. View System Analytics & Utilization (Streams API)
7. Export Audit & Reservation Report to CSV
8. View Live Audit Log Trail
9. Run Concurrency Stress Demonstration
0. Exit
```



## 9.2 Streams Analytics & Concurrency Test

```
--- SYSTEM ANALYTICS & UTILIZATION (STREAMS API) ---
Reservations Breakdown by Status:
• CONFIRMED : 2 bookings
Booked Hours by Facility Category:
• Computer Lab : 3.00 total hours
• Auditorium : 2.00 total hours

--- CONCURRENCY RACE CONDITION TEST ---
Simulating 2 threads booking the SAME slot simultaneously...
>> [Thread-1 GDSC] SUCCESS: Acquired slot -> RES-1003
>> [Thread-2 Robotics] REJECTED: CONFLICT: Slot reserved by #RES-1003
>> Double-booking prevented by synchronization monitor!
```

## 10. Testing Approach & Test Matrix

| Test ID | Scenario                      | Expected Outcome               | Result |
|---------|-------------------------------|--------------------------------|--------|
| TC-01   | Valid Booking Creation        | Confirmed Reservation returned | PASS   |
| TC-02   | Clashing Slot Rejection       | SlotConflictException thrown   | PASS   |
| TC-03   | Role Quota Enforcement        | QuotaExceededException thrown  | PASS   |
| TC-04   | Cancellation & Slot Release   | Slot freed and re-bookable     | PASS   |
| TC-05   | Multi-Threaded Race Condition | Exactly 1 success, 1 conflict  | PASS   |

```

STARTING AUTOMATED TEST RUNNER: CampusResourceHub

Test 1: Valid Booking Creation ... PASSED
Test 2: Double Booking Conflict ... PASSED (SlotConflictException)
Test 3: Quota Limit Enforcement ... PASSED (QuotaExceededException)
Test 4: Cancellation & Re-availability ... PASSED
Concurrency Test: 2 Threads same slot ... PASSED (1 Success, 1 Conflict)
ALL TEST SUITES EXECUTED SUCCESSFULLY! (5/5 PASSED)
```

## 11. Challenges Faced & Practical Solutions

---

During the development of this project, I ran into several issues that took time and experimentation to resolve:

### 1. Dealing with Race Conditions in Concurrent Requests:

When I first tested my booking logic with two threads, both of them managed to book the same auditorium slot. It took me a while to understand why—both threads were reading the slot as available before either of them could write their reservation. I spent time reading about Java concurrency primitives and eventually learned about lock striping. Instead of locking the entire system with one giant synchronized block (which would have been slow), I created a separate lock object for each resource ID using `ConcurrentHashMap.computeIfAbsent()`. This way, if two clubs are booking different rooms, they don't block each other at all—only threads going after the exact same venue need to wait.

### 2. Getting Priority Preemption Right Without Corrupting State:

I wanted faculty bookings to override student club ones when there's a clash. But in my first attempt, I was cancelling the old booking and creating the new one in separate steps, which briefly left the slot in a weird state. To fix this, I moved both operations—cancelling the club reservation and creating the faculty one—inside the same synchronized block. This way, the slot never appears "empty" between the cancel and the new booking.

### 3. Making It Work on Any Computer Without Setup Hassle:

In a previous project I tried using MySQL, and the evaluator couldn't run it because they didn't have the database installed. This time I deliberately avoided external dependencies. Everything runs with just `javac` and `java`. For persistence, I used Java's built-in Object Serialization. For testing, instead of pulling in JUnit via Maven, I wrote a simple custom test runner with assertion checks.

## 12. Learnings & Key Takeaways

---

Working on this project gave me a much deeper understanding of Java than I had from just studying lecture slides. Here are the main things I took away:

- **OOP Actually Makes Sense in Practice:** Before this project, abstract classes and interfaces felt like unnecessary complexity. But when I needed to add a new venue type (`ConferenceRoom`), I just extended `CampusResource` and implemented two methods. Everything else—booking, analytics, display—worked automatically because of polymorphism. That was a real "aha" moment for me.
- **Concurrency Is Harder Than It Looks:** I thought adding `synchronized` to a method would be enough. It wasn't. I learned about lock granularity, the importance of `CountDownLatch` for firing threads simultaneously in tests, and why `ConcurrentHashMap` is preferred over wrapping a regular `HashMap` with `synchronized` blocks.
- **Streams API Changed How I Think About Data:** Computing analytics using `filter().collect(groupingBy())` was so much cleaner than the nested for-loops I was writing before. It also helped me appreciate lambda expressions and method references.
-

**Good Exception Design Saves Debugging Time:** Instead of printing generic error messages, having named exceptions like `QuotaExceededException` made it immediately clear what went wrong during testing. I didn't have to add print statements everywhere to trace bugs.

## 13. Future Enhancements

---

- **Graphical Interface:** Building a desktop JavaFX or web-based Spring Boot frontend to replace the CLI.
- **Database Integration:** Connecting to SQLite or PostgreSQL via JDBC for proper relational storage.
- **Notification System:** Using `ScheduledExecutorService` to dispatch email or SMS alerts upon booking preemption.
- **Access Token Generation:** Generating QR codes upon booking approval for venue entry verification.

## 14. References

---

1. Bloch, Joshua. *Effective Java (3rd Edition)*. Addison-Wesley Professional, 2018.
2. Schildt, Herbert. *Java: The Complete Reference (12th Edition)*. McGraw-Hill Education, 2021.
3. Oracle Java Documentation: *Java Platform SE & Concurrency Utilities API Specification*.
4. VIT Bhopal University: *CSE1007 Programming in Java Course Syllabus and Project Guidelines*.